

对象序列化之四



自行定制序列化过程

北京理工大学计算机学院
金旭亮

不序列化某些字段



通过在字段前面加上**transient**关键字，可以让此字段不参与序列化。

```
class PersonIgnoreage implements java.io.Serializable {  
    private String name;  
    private transient int age;  
    ...  
}
```

```

public class TransientTest {
    4 usages
    private static final String dataFileName = "transient.dat";

    public static void main(String[] args)
        throws IOException, ClassNotFoundException {
        try (ObjectOutputStream oos = new ObjectOutputStream(
            new FileOutputStream(dataFileName));
            ObjectInputStream ois = new ObjectInputStream(
                new FileInputStream(dataFileName))) {
            PersonIgnoreage per = new PersonIgnoreage(name: "张三", age: 46);
            //系统会将对象转换为字节序列并输出
            oos.writeObject(per);
            System.out.println("向文件" + dataFileName + "中写入对象:" + per);
            PersonIgnoreage p = (PersonIgnoreage) ois.readObject();
            System.out.println("向文件" + dataFileName + "中读取对象:" + p);
        }
    }
}

```

从输出可以清楚地看到，age字段没有参与序列化，因此，反序列化时，得到了一个默认值0。

示例：TransientTest.java

Run: TransientTest x







"C:\Program Files\Java\jdk-17\bin\java.exe"

向文件transient.dat中写入对象:张三有46岁

向文件transient.dat中读取对象:张三有0岁

Process finished with exit code 0

控制序列化过程

如果需要完全自定义序列化过程，可以让对象实现 **Serializable** 接口，并添加以下序列化和反序列化方法。

```
private void writeObject(java.io.ObjectOutputStream out)
    throws IOException
{
    //...
}

private void readObject(java.io.ObjectInputStream in)
    throws IOException, ClassNotFoundException
{
    //...
}
```

示例：CustomSerialize.java



自定义序列化示例-1

明确定义如何从流中
读取和写入数据。

```
class MyPerson implements Serializable {  
    6 usages  
    private String name;  
    6 usages  
    private int age;  
    1 usage  
    public MyPerson(String name, int age) {...}  
    //提供编译器的语法检查 (可选)  
    @Serial  
    private void writeObject(ObjectOutputStream out)  
        throws IOException {  
        out.writeObject(new StringBuffer(name).reverse());  
        out.writeInt(age);  
    }  
    @Serial  
    private void readObject(ObjectInputStream in)  
        throws IOException, ClassNotFoundException {  
        this.name = ((StringBuffer) in.readObject()).toString();  
        this.age = in.readInt();  
    }  
    @Override  
    public String toString() {  
        return name + "有" + age + "岁";  
    }  
    getter and setter  
}
```

自定义序列化示例-2



```
public class CustomSerialize {  
    4 usages  
    private static final String objDataFileName = "customSerializeObj.dat";  
  
    public static void main(String[] args) {  
        try (ObjectOutputStream oos = new ObjectOutputStream(  
            new FileOutputStream(objDataFileName));  
            ObjectInputStream ois = new ObjectInputStream(  
                new FileInputStream(objDataFileName)))  
        ) {  
            MyPerson per = new MyPerson( name: "张三", age: 23);  
            oos.writeObject(per);  
            System.out.println("向文件" + objDataFileName + "中写入对象: " + per);  
            MyPerson p = (MyPerson) ois.readObject();  
            System.out.println("从文件" + objDataFileName + "中读入对象: " + p);  
        } catch (Exception ex) {  
            ex.printStackTrace();  
        }  
    }  
}
```

从程序输出结果中看，确实按照“逆序”方式将MyPerson对象的名字字段值写入到了文件中。

Run: CustomSerialize x

"C:\Program Files\Java\jdk-17\bin\java.exe" "
向文件customSerializeObj.dat中写入对象: 张三有23岁
从文件customSerializeObj.dat中读入对象: 三张有23岁

Process finished with exit code 0

“改头换面”的序列化



阅读示例ReplaceTest.java源码，看看此示例如何完成一个“神奇”的转换，将一个Person对象序列化为ArrayList。

```
ReplaceTest x
"C:\Program Files\Java\jdk-17\bin\java.exe"
对象: "张三有34岁"已写入到replace.dat文件中
从replace.dat文件中读出: [张三, 34]
Process finished with exit code 0
```

要点:

1. ReplacedPerson类要实现Serializable接口。
2. 要添加一个writeReplace方法。
3. Java在序列化某个对象时，总是先调用writeReplace方法再调用writeObject方法。

应用实例：利用序列化技术实现对象克隆



基本方法：

让对象实现Cloneable和Serializable接口，
利用ByteArrayOutputStream实现克隆。

示例：SerialCloneTest.java

对象克隆实例分析-1



定义一个基类，让它实现 Cloneable 和 Serializable 接口，然后，让那些需要克隆的对象从它继承而来。

在基类的 clone 方法中，实例化一个内存字节数组输出流，将要克隆的对象序列化到内存流中，之后，再调用 readObject() 方法反序列化，得到克隆后的对象。

```
public class SerializableCloneableBase implements Cloneable, Serializable {  
    public Object clone() throws CloneNotSupportedException {  
        Object clone = super.clone();  
        try (var bout = new ByteArrayOutputStream();  
             ObjectOutputStream out = new ObjectOutputStream(bout);  
        ) {  
            out.writeObject(this); //将自身状态序列化到流中  
            try (var bin = new ByteArrayInputStream(bout.toByteArray());  
                 ObjectInputStream in = new ObjectInputStream(bin)) {  
                //基于流反序列化，得到一个克隆对象  
                clone = in.readObject();  
            }  
        } catch (Exception e) {  
            System.out.println(e);  
        }  
        //向外界返回克隆结果  
        return clone;  
    }  
}
```

对象克隆实例分析-2



```
public class ClonablePerson
    extends Serializable {
    4 usages
    private String name;
    4 usages
    private int age;
    1 usage
    public ClonablePerson(String name, int age) {...}

    "getter and setter"

    @Override
    public String toString() {
        return name + "有" + age + "岁";
    }
}
```

```
public class SerialCloneTest {
    public static void main(String[] args)
        throws CloneNotSupportedException {
        var zhangShan = new ClonablePerson(name: "张三", age: 20);
        //实现对象克隆
        ClonablePerson zhangShan2 = (ClonablePerson) zhangShan.clone();
        //输出克隆对象的内容
        System.out.println(zhangShan);
        System.out.println(zhangShan2);
        //这两个对象是不是独立的?
        System.out.println(zhangShan == zhangShan2);
    }
}
```

从运行结果中可以看到，ClonablePerson对象成功地克隆了一个内容完全一样的副本，并且两者是独立无关的。

```
Run: SerialCloneTest x
"C:\Program Files\Java\jdk-17\bin\java.exe"
张三有20岁
张三有20岁
false
Process finished with exit code 0
```

开发技能训练项目：对象数据库



有一个MyClass类，其中包含id和name两个字段。



编写一个程序，可以将多个MyClass对象写入到MyClassDB.dat文件中。



程序支持按id或name值查找对象，并且允许在指定的位置插入和删除特定的MyClass对象。